
Lambdas

Le problème

Comment régler le problème de la verbosité des classes anonymes ?

- Création d'un objet,
- implémentant une interface,
- souvent avec une seule méthode.

Verbeux : 4 lignes pour une instruction !

- une ligne pour la déclaration de classe,
- une ligne pour la déclaration de méthode,
- puis les instructions de la méthode (souvent une seule),
- puis fermer les deux paires d'accolades

Lambda et interface fonctionnelle

Interface fonctionnelle

Une interface est dite **fonctionnelle** si elle contient une seule méthode sans implémentation par défaut (et un nombre arbitraire de méthodes par défaut).

Lambda

Une **lambda** est une **expression**, utilisant une syntaxe spéciale, pour définir une classe anonyme implémentant une interface fonctionnelle.

Exemple d'interface fonctionnelle

Une interface fonctionnelle :

```
@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);
}
```

Utilisation avec des lambdas :

```
List<Car> vaillantes =
    carsDealers.select(
        car -> "Vaillante".equals(car.getBrand())
    );
List<Car> cheapCars =
    carsDealers.select(car -> car.getPrice() <= 15000);
```

Sucre syntaxique

Les lambdas sont une **utilité syntaxique** :

- ne permettent rien de plus que certaines formes de classes anonymes,
- peuvent toujours être remplacées par une classe anonyme équivalente.

On parle de **sucre syntaxique** : le compilateur traduit automatiquement la fonctionnalité vers la syntaxe verbeuse.

Les enregistrements (**record**) sont un autre exemple de sucre syntaxique.

Exemple de traduction

```
car -> car.getPrice();
```

est traduit en :

```
new ToDoubleFunction<Car>() {  
    public double applyAsDouble(Car car) {  
        return car.getPrice();  
    }  
}
```

Exemple de traduction

```
bound -> {  
    int product = 1;  
    for (int n = 0; n <= bound; n++) {  
        product = product * bound;  
    }  
    return product;  
}
```

est traduit en :

```
new IntUnaryOperator() {  
    public int apply(int bound) {  
        product = 1;  
        for (int n = 0; n <= bound; n++) {  
            product = product * bound;  
        }  
        return product;  
    }  
}
```

Syntaxe des lambdas

Deux syntaxes :

```
(arg1, ..., argN) -> expression(arg1, ..., argN)  
(arg1, ..., argN) -> { bloc d'instructions avec return }
```

- Si un seul paramètre, les parenthèses peuvent être omises.
- Si pas de paramètre, on le note ().
- Il n'est en général pas nécessaire de donner le type des paramètres.
- Si on indique le type d'un paramètre, on doit indiquer le type de tous les paramètres (**c'est tout ou rien**).

Références de méthode (I)

Souvent, l'expression lambda a une des formes :

```
obj -> obj.method();  
value -> obj.method(value);  
value -> MyClass.staticMethod(value);
```

Exemples :

```
cars.forEach(car -> System.out.println(car));  
ToIntFunction<Car> toProductionYear =  
    car -> car.getProductionYear();  
IntProducer randomInts = () -> gen.nextInt();
```

Références de méthode (II)

On peut alors utiliser une référence de méthode pour simplifier la notation :

```
MyClass::method // obj -> obj.method()  
obj::method     // value -> obj.method(value)  
MyClass::method // value -> MyClass.method(value)  
MyClass::new     // value -> new MyClass(value)
```

Exemples :

```
cars.forEach(System.out::println);  
IntProducer randomInts = gen::nextInt;  
Comparator<Car> compareCarByProductionYear =  
    Comparator.comparing(Car::getProductionYear);
```

Analyse de la solution avec lambdas

- Peu verbeux.
- Autorise la création de **valeurs fonctionnelles**, pour passer des fonctions en paramètres, ou retourner des fonctions.
- Cache le fait que les fonctions sont représentées en Java par des objets à une seule méthode.

Limites :

- Toujours pas composable.

La solution au prochain cours...

Classes anonymes

Filtrer une liste par critère

Utilisation d'une interface, pour généraliser plusieurs critères :

```
public interface Predicate<T> {  
    boolean test(T value);  
    ...  
}  
  
public List<Car> select(Predicate<Car> property) {  
    List<Car> selection = new ArrayList<>();  
    for (Car car : cars) {  
        if (property.test(car)) selection.add(car);  
    }  
    return selection;  
}  
}
```

Réalisation des critères

```
public class HasBrand implements Predicate<Car> {
    private final String expectedBrand;
    public HasBrand(String brand) { expectedBrand = brand }
    public boolean test(Car car) {
        return car.brand.equals(expectedBrand);
    }
}

public class HasMaxPrice implements Predicate<Car> {
    private final int maxPrice;
    public HasMaxPrice(int maxPrice) { this.maxPrice = maxPrice; }
    public boolean test(Car car) {
        return car.getPrice() <= maxPrice;
    }
}

// Exemples d'utilisation
List<Car> vaillantes = carDealer.select(new HasBrand("Vaillante"));
List<Car> cheapCars = carDealer.select(new HasMaxPrice(15000));
```

Analyse de cette solution

- interface Predicate, représente un critère de test,
- chaque critère possible requiert une classe implémentant Predicate,
- méthode select prenant un Predicate,
- select prend donc une **fonction** en paramètre !

Limites :

- Verbeux,
- nécessite d'écrire une classe complète pour chaque test.
- Toujours pas composable.

Classes anonymes

On peut éviter de devoir définir une classe pour chaque critère de sélection, avec les **classes anonymes**.

Création d'un objet de l'interface Comparator :

```
Comparator<Double,Double> doubleComparator =  
    new Comparator() {  
        public int compare(Double x, Double y) {  
            double diff = Math.abs(x - y);  
            return (diff < EPSILON) ? 0:  
                (x > y) ? 1:  
                    -1;  
        }  
    };
```

Classes anonymes (interface)

Syntaxe :

```
new InterfaceName() {  
    public type1 method1(Arg1 arg1) { ... }  
    public type2 method2(Arg2 arg2) { ... }  
    ...  
}
```

Peut contenir :

- des méthodes (privées, publiques, ...),
- des propriétés (final, privées, ...),
- pas de constructeur.

Classes anonymes (extension)

On peut aussi créer une **extension** anonyme d'une classe.

Syntaxe :

```
new SuperClassName(constructorArgs) {  
    ... // méthodes et propriétés (dont redéfinition)  
}
```

Exemple :

```
Point2D origin =  
    new Point2D(0,0) {  
        @Override public String toString() {  
            return "origin";  
        }  
    };
```

Classes anonymes

- l'instanciation d'une classe anonyme est une **instruction** (elle a un effet) et une expression (elle produit une valeur) !
- Définition et instanciation simultanée d'une classe,
- La classe est instanciée une seule fois,
- La classe ne porte pas de nom, donc **elle n'est pas réutilisable** ailleurs.
- L'objet créé **implémente** une interface ou **étend** une autre classe.
- Dans le cas d'une extension, les paramètres du constructeur de la classe parent sont donnés entre les parenthèses de la déclaration de la classe anonyme.

Limitation des classes anonymes

Une classe anonyme

- peut déclarer des méthodes et des propriétés,
- ne peut pas déclarer d'interfaces,
- ne peut rien posséder de statique (sauf des constantes),
- **n'a pas de constructeur** (mais peut avoir un initialiseur),
- peut utiliser des propriétés de l'objet, et des variables de la méthode qui la définit **seulement si** elles sont déclarées **final** !

Application à la sélection

```
// Exemples d'utilisation
List<Car> vaillantes =
    carDealer.select(new Predicate<Car>() {
        public boolean test(Car car) {
            return "Vaillante".equals(car.getBrand());
        }
    });

List<Car> cheapCars =
    carDealer.select(new Predicate<Car>() {
        public boolean test(Car car) {
            return car.getPrice() <= 15000;
        }
    });
```

Analyse de cette solution

Avec des classes anonymes :

- évite de multiplier les petites classes,
- moins verbeux (disparition du constructeur, des propriétés),
- mais casse la propriété de la hiérarchie des programmes (classe, méthodes, instructions), puisque désormais une méthode peut contenir une instruction définissant une classe.

Limites :

- Toujours pas composable.